

Parsing USCF Chess Crosstables in R

<https://gemini.google.com/app/2223fab125e9c2ed>

User prompt: I have a text document (hosted at a url) that I would like to read in with R. It is formatted like below where bars are the delimiters after a few header lines:

```
----- Pair | Player Name
|Total|Round|Round|Round|Round|Round|Round|Round| Num | USCF ID / Rtg (Pre->Post) | Pts | 1 | 2 | 3 | 4
| 5 | 6 | 7 | ----- 1 | GARY HUA |6.0 |W 39|W
21|W 18|W 14|W 7|D 12|D 4| ON | 15445895 / R: 1794 ->1817 |N:2 |W |B |W |B |W |B |W |
----- 2 | DAKSHESH DARURI |6.0 |W 63|W
58|L 4|W 17|W 16|W 20|W 7| MI | 14598900 / R: 1553 ->1663 |N:2 |B |W |B |W |B |W |B |
----- 3 | ADITYA BAJAJ |6.0 |L 8|W 61|W
25|W 21|W 11|W 13|W 12| MI | 14959604 / R: 1384 ->1640 |N:2 |W |B |W |B |W |B |W |
----- 4 | PATRICK H SCHILLING |5.5 |W 23|D
28|W 2|W 26|D 5|W 19|D 1| MI | 12616049 / R: 1716 ->1744 |N:2 |W |B |W |B |W |B |B |
----- 5 | HANSHI ZUO |5.5 |W 45|W 37|D
12|D 13|D 4|W 14|W 17|
```

Response: This format is a standard USCF (United States Chess Federation) tournament crosstable export. Notice that each player's entry spans **two physical lines**:

1. **Line 1:** Contains the Pair Number, Player Name, Total Points, and results for each round (e.g., W 39, D 12).
2. **Line 2:** Contains the state/region code, USCF ID, pre/post ratings, and color allocations (White/Black) for each round.
3. **Separators:** Rows are divided by lines of dashes (-----).

Because each player record is split across two rows and interspersed with separator lines, a standard `read.csv()` or `read.table()` will break. The most robust way to parse this in R is to read the file as raw lines, filter out the separator rows, pair up the consecutive player lines, split by the `|` delimiter, and bind them into a clean data frame.

Here is a complete, step-by-step R solution using tidyverse (or base R alternatives):

Step 1: Read and Clean the Lines

```
library(stringr)
library(dplyr)
library(tidyr)
```

```
# Replace with your actual URL or local file path
url <- "YOUR_URL_TO_THE_TEXT_FILE_HERE"
```

```
# Read all lines from the URL
raw_lines <- readLines(url)

# 1. Remove rows that are entirely dashed separators
content_lines <- raw_lines[!str_detect(raw_lines, "^\\s*---")]

# Remove header lines if they appear at the top (usually the first 3 lines)
# Adjust the index based on your exact file structure
content_lines <- content_lines[3:length(content_lines)]
```

Step 2: Reshape the Two-Line Records into Single Rows

Since every player takes up two consecutive lines, we can use grouping tricks to combine them into a single wide string before splitting by |.

```
# Create a grouping variable where every pair of lines shares the same ID
player_groups <- rep(1:(length(content_lines) / 2), each = 2)

# Combine the two lines for each player into a single string
combined_records <- tapply(content_lines, player_groups, paste0, collapse = " | ")

# Now split by '|' and trim whitespace from each cell
split_matrix <- str_split(combined_records, "\\|")
cleaned_data <- lapply(split_matrix, function(row) str_trim(row))

# Convert list to a data frame
df <- as.data.frame(do.call(rbind, cleaned_data), stringsAsFactors = FALSE)
```

Step 3: Assign Column Names

Based on your header layout, you can assign descriptive column names to the resulting data frame:

```
# Depending on how many rounds your tournament has, set names accordingly.
# For 7 rounds, there will be 11 columns total.
colnames(df) <- c(
  "Pair_Num_Or_State",
  "Player_Name_Or_ID",
  "Total_Pts_Or_Rtg",
  "R1", "R2", "R3", "R4", "R5", "R6", "R7", "Extra"
)

head(df)
```

Alternative: Using Specialized Parsers

If you plan to do extensive chess data analysis, check out the **chessutils** or **rchess** packages on CRAN or GitHub, though custom text-wrangling via `readLines()` and `stringr` as shown above gives you total control over messy USCF text formats.

User prompt: I have multiple columns where I would like to replace any cell that does not contain a "W", "L", or "D" with a NaN

Response: In R, missing values are typically represented by NA rather than NaN (which stands for *Not a Number* in numeric contexts).

You can accomplish this using dplyr with mutate() and across(), or in Base R.

Option 1: Using tidyverse / dplyr (Recommended)

If you know the specific column names:

```
library(dplyr)
library(stringr)

# Replace values across specific columns (e.g., R1 through R7)
df <- df %>%
  mutate(across(c(R1, R2, R3, R4, R5, R6, R7), ~ ifelse(str_detect(., "[WLD]"), ., NA)))
```

If you want to target all round columns dynamically (e.g., any column starting with "R"):

```
df <- df %>%
  mutate(across(starts_with("R"), ~ ifelse(str_detect(., "[WLD]"), ., NA)))
```

Option 2: Base R

Using grepl() across a selection of columns by name or index:

```
cols <- c("R1", "R2", "R3", "R4", "R5", "R6", "R7") # Adjust column names as needed

df[cols] <- lapply(df[cols], function(x) {
  ifelse(grepl("[WLD]", x), x, NA)
})
```

How It Works

- [WLD] is a regular expression pattern that checks if the cell contains the letter **W**, **L**, or **D**.
- ifelse(...) evaluates each cell: if the letter is present, it keeps the original string; otherwise, it assigns NA.

User prompt: for each row, I have seven columns labeled R1_opp_name, R2_opp_name, ..., R7_opp_name. In those columns is "num" corresponding to the number row or another opponent. I would like to create another set of columns R1_opp_name_score that shows the "pre-rating" value of the referenced row in R1_opp_name. If the R1_opp_name, is NA, then the R1_opp_name_score should also be

NA.

Response: To look up a value from another row in the same data frame based on a row number stored in a column, you can index your vector of pre-ratings directly using those row indices.

Here is how to create the `_score` columns across all seven rounds cleanly using `dplyr` or Base R:

Option 1: Using `dplyr` and `across()`

```
library(dplyr)
```

```
# Assuming 'pre_rating' is the column containing each player's pre-rating numeric value
df <- df %>%
  mutate(
    across(
      starts_with("R") & ends_with("_opp_name"),
      ~ pre_rating[.],
      .names = "{.col}_score"
    )
  )
```

Option 2: Base R Loop

```
# Define the opponent columns
opp_cols <- paste0("R", 1:7, "_opp_name")

# Loop through each opponent column and pull the matching pre_rating
for (col in opp_cols) {
  new_col <- paste0(col, "_score")
  df[[new_col]] <- df$pre_rating[df[[col]]]
}
```

How It Works:

- **Vectorized Lookup (`pre_rating[.]`):** In R, indexing a vector with numbers (e.g., `pre_rating[c(39, 21, NA)]`) automatically returns the pre-rating values corresponding to those row positions.
 - **NA Handling:** If an opponent column contains NA, vector indexing naturally returns NA for that lookup value without requiring extra if or ifelse statements.
-